ICLR 2026

GenCP: Towards Generative Modeling Paradigm of Coupled Physics

Tianrun Gao[*†1,2] **Haoren Zheng**[*†1,3] **Wenhao Deng**[*1] **Haodong Feng**[1]
Tao Zhang[†1,4] **Ruiqi Feng**[1] **Qianyi Chen**[‡1] **Tailin Wu**[‡1]
[1]School of Engineering, Westlake University;
[2]Department of Geotechnical Engineering, Tongji University;
[3]Global College, Shanghai Jiao Tong University;
[4]State Key Laboratory of Advanced Nuclear Energy Technology, Nuclear Power Institute of China;
{gaotianrun, chenqianyi, wutailin}@westlake.edu.cn

2026.4.16 张欣宇

- Coupled Physics 多物理场耦合问题
- Generative Model

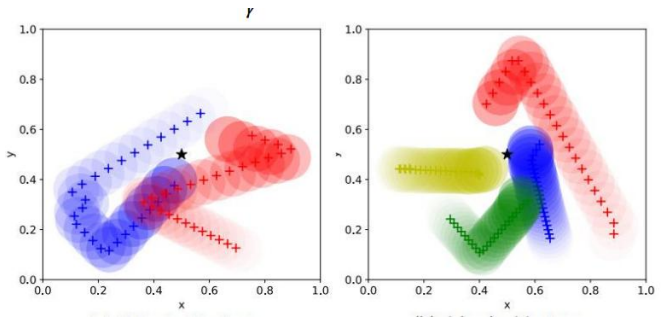
大系统拆分成子系统——复杂关系拆分成简单关系

上次讲的论文

CinDM

COMPOSITIONAL GENERATIVE INVERSE DESIGN

Tailin Wu[1*†], Takashi Maruyama[2*], Long Wei[1*], Tao Zhang[1*], Yilun Du[3*], Gianluca Iaccarino[4], Jure Leskovec[5]
[1]Dept. of Engineering, Westlake University, [2]NEC Laboratories Europe,
[3]Dept. of Computer Science, MIT, [4]Dept. of Mechanical Engineering, Stanford University,
[5]Dept. of Computer Science, Stanford University



团队的另一篇论文，GenCP在M2PDE进行改进+详细的理论分析

M2PDE: Compositional Generative Multiphysics and Multi-component PDE Simulation

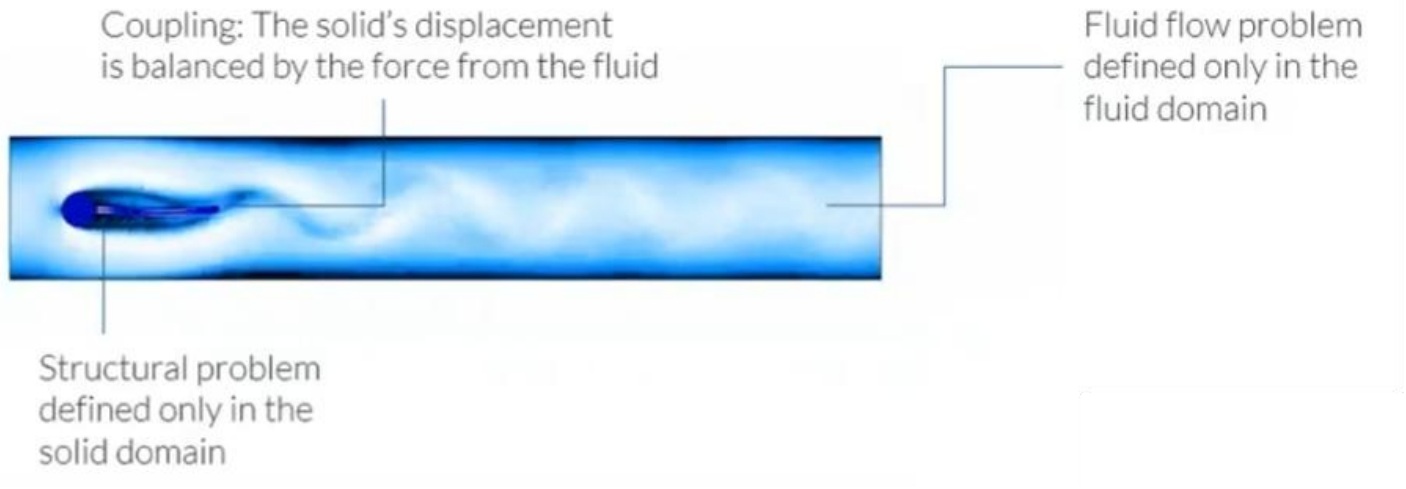
Tao Zhang [1] Zhenhai Liu [1] Feipeng Qi [1] Yongjun Jiao [1] Tailin Wu [2]

Preliminaries——多物理场耦合问题直观描述

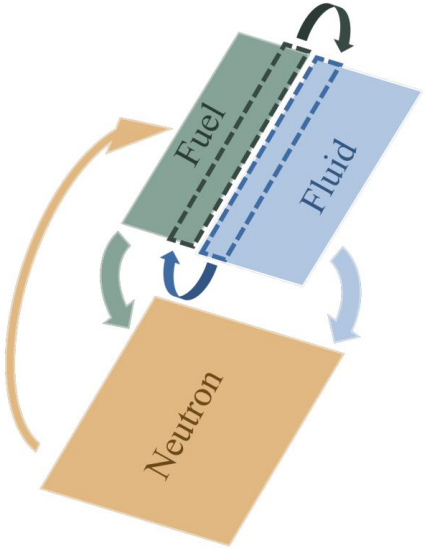
定义定义：涉及多个物理过程相互作用的系统，各物理场之间存在依赖关系

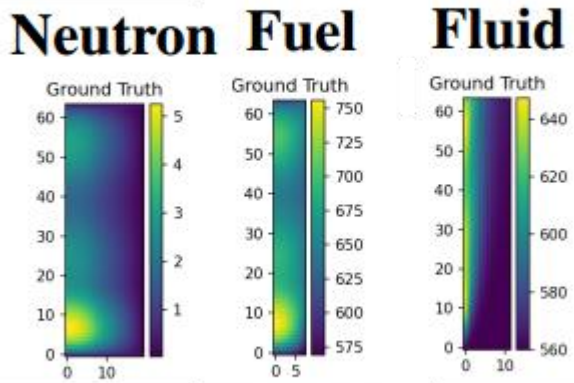
经典问题：Fluid Structure Interaction：

- **流体影响固体**：流体流动时会产生压力（力），这个力作用在固体上，会推动固体变形或移动。
- **固体影响流体**：固体变形后，会改变流体流动的区域边界，从而改变流体的速度场和压力场。



更复杂的问题：核心热耦合：





传统数值方法——紧耦合

（以耦合模式做分类）

整个物理场同时求解

$$\begin{cases} A\vec{e} + \alpha\vec{T} = \vec{a} \\ \beta\vec{e} + B\vec{T} = \vec{b} \end{cases}$$

两个相互耦合的变量

$$\begin{bmatrix} A & \alpha \\ \beta & B \end{bmatrix} \begin{bmatrix} \vec{e} \\ \vec{T} \end{bmatrix} = \begin{bmatrix} \vec{a} \\ \vec{b} \end{bmatrix}$$

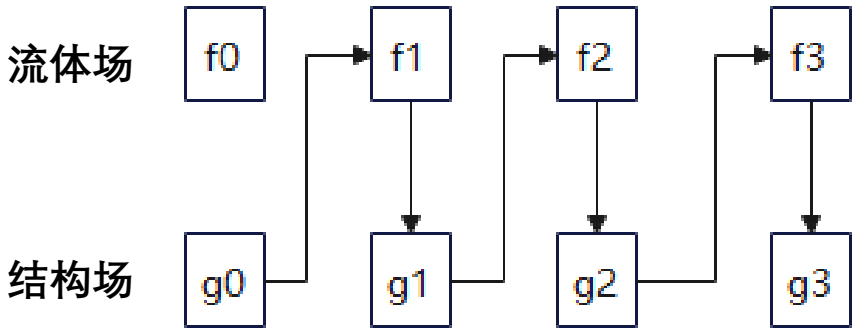
将所有物理场的方程组装成一个整体系统，同时求解。
优点：高精度，高稳定性
缺点：计算代价高，适用于小规模问题

典型方法：矩阵方法（如**有限元法**或**有限差分法**）：将每个物理场方程的离散化表示成矩阵形式，并将它们组合成一个大的联合矩阵

传统数值方法——松耦合

将系统分解为子问题，顺序或迭代求解。

Picard迭代

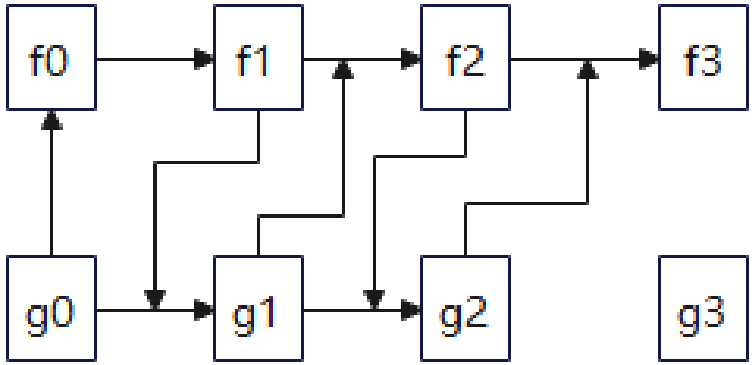


Picard 迭代的过程示意：

1. g_0 固定，计算 f_1
2. f_1 固定，计算 g_1
3. g_1 固定，计算 f_2
4. f_2 固定，计算 g_2
5. 继续交替迭代，直到收敛。

高效

Operator Splitting



Lie-Trotter为例

1. **初始化**：给定初始值 f_0 和 g_0。
2. **第一步（更新 f）**：根据 g_0 更新 f_1，但 g_0 保持不变。
3. **第二步（更新 g）**：根据 f_1 更新 g_1，但 f_1 保持不变。
4. **继续独立更新**：每个物理场的更新只依赖于另一个场在当前时间步的固定值，而不是交替迭代的结果。

链式：完全依赖上一步的解（强制更新）

优点：

- **高精度**：通过迭代修正每个物理场的解适用于耦合较强的多物理场问题。

缺点：

- **高耦合导致计算成本高**：每次迭代都需要处理多个物理场的相互影响，尤其是当耦合非常强时，迭代次数可能很多，因此计算资源消耗较大。

优点：

每次迭代时都只依赖于**固定的另一个物理场**，而不是更新后的结果

- 灵活

缺点：

- 不适用于强耦合问题
- 依赖于分裂策略

理论保证：数值分析里面的经典迭代工具，可接受的误差范围上享有理论保证

传统数值方法→代理模型方法→生成模型方法

高保真但昂贵

低高保真但迅速

Surrogate Model

紧耦合

松耦合

Generative Model

紧耦合

松耦合

松耦合->紧耦合　（数据获取难度）
紧耦合（f0，g0）->（f1，f2）
松耦合（f0，g0）->f1
数据驱动的代理模型，使用紧耦合需要知道全部的耦合数据（数据获取难度大）

松耦合只需要知道子物理场的数据及条件，数据采集成本低

代理模型->生成模型
代理模型缺点：

- 捕捉高频、高维和随机行为的能力有限，难以处理复杂的**时空动态**

生成模型做多物理场生成现状：
1.多以紧耦合 直接全场生成为主

2. （重要的baseline：G2PDE）
松耦合有效但是缺乏收敛性的理论保证

论文目标：松耦合 +生成模型（高保真） + 高效采样 +理论一致

Operator Splitting

GenCP: Towards Generative Modeling Paradigm of Coupled Physics

论文目标：松耦合 +生成模型（高保真） + 高效采样 +理论一致

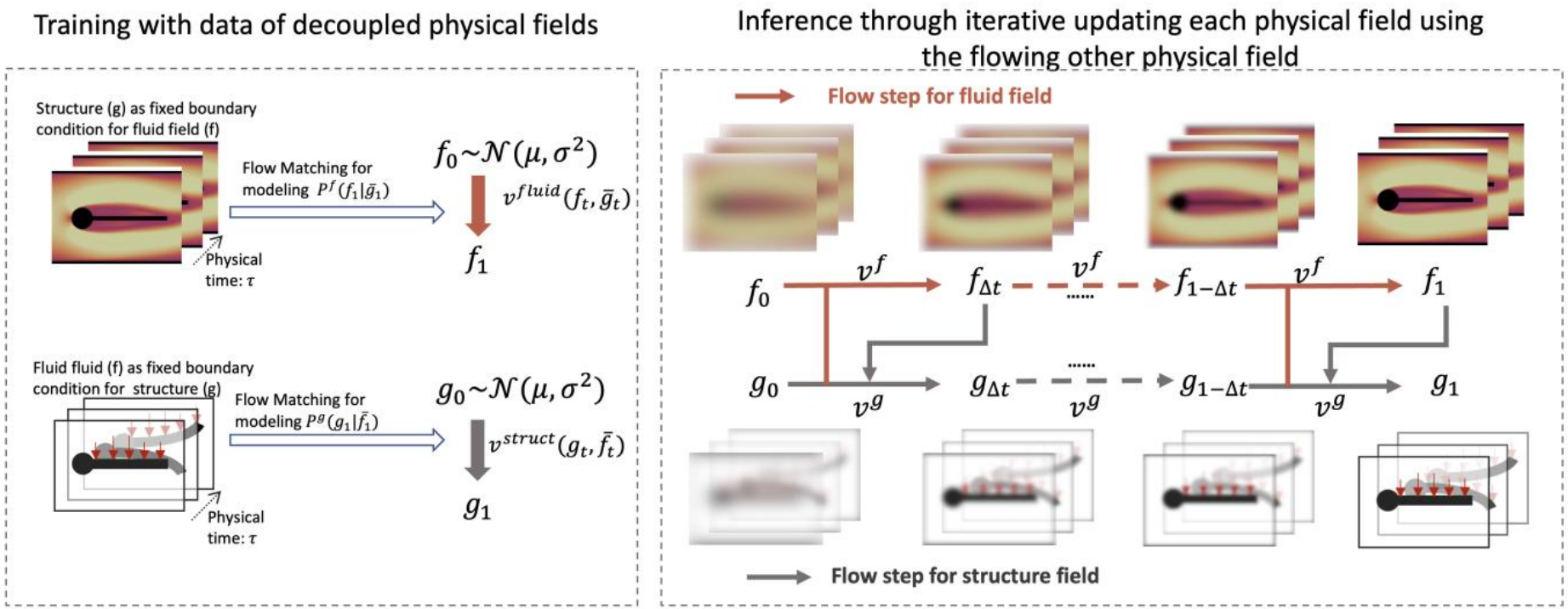
生成模型选择　　迭代策略

仅用解耦的物理场训练+Functional Flow Matching + Operator Splitting -> 耦合物理场求解-> 详细的误差分析

?　　?

结构场
（f1，g1）->g2

流体场
（f1，g1）->f2



问题1：Operator Splitting高效在了哪里

问题2：什么是Functional Flow Matching（FFM），为什么用FFM，FM不行吗

Flow Matching

$$\frac{d}{dt}p_t(x) + \mathrm{div}(p_t(x)v_t(x)) = 0,$$

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,q(x_1),p_t(x|x_1)}\left\|v_t(x) - u_t(x|x_1)\right\|^2,$$

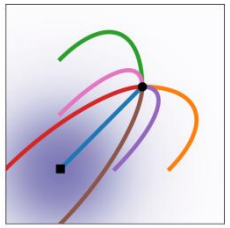
对于数据点 $x_1 \in \mathbb{R}^n$，定义条件概率密度路径 $p_t(x \mid x_1)$:

$$p_t(x \mid x_1) = \mathcal{N}\left(x \mid \mu_t(x_1), \sigma_t^2 I\right)$$

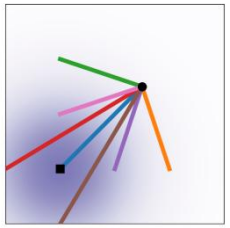
$$u_t(x|x_1) = \frac{\sigma_t'(x_1)}{\sigma_t(x_1)}(x - \mu_t(x_1)) + \mu_t'(x_1).$$

定义最优传输路径 $\mu_t(x) = tx_1$, and $\sigma_t(x) = 1 - (1 - \sigma_{\min})t.$

σmin=0时 $u_t(x) = x_1 - x$



Diffusion



OT

GenCP 使用Functional Flow Matching

生成路径在函数空间$\mathcal{F}$ (无限维函数空间)

$$z_f \sim \pi_{\mathcal{F}},$$

$$f_t = (1-t)z_f + tf_1$$

$$\frac{df_t}{dt} = v_f = f_1 - z_f.$$

$$\mathcal{L}_f(\theta_f) = \mathbb{E}_{t,(f_1,\bar{g})\sim\mathcal{D}_f,\, z_f, z_g}\left[\|v_f - \hat{v}_f(f_t, g_t, t; \theta_f)\|_{\mathcal{F}}^2\right],$$

$$\mathcal{L}_g(\theta_g) = \mathbb{E}_{t,(\bar{f},g_1)\sim\mathcal{D}_g,\, z_f', z_g'}\left[\|v_g - \hat{v}_g(f_t, g_t, t; \theta_g)\|_{\mathcal{G}}^2\right].$$

（加入了其它场状态作为条

传输路径实际代码：

Algorithm 1 Coupled Sampling via Lie–Trotter Splitting

```
Input: initial u_0 = (f_0, g_0), trained operators v̂_f, v̂_g, steps N
u ← u_0
for k = 0 to N − 1 do
    t ← k/N
    f ← f + τ v̂_f(f, g, t; θ_f)
    g ← g + τ v̂_g(f, g, t; θ_g)
    u ← (f, g)
end for
return u
```

GenCP: TOWARDS GENERATIVE MODELING PARADIGM OF COUPLED PHYSICS

论文目标：松耦合 +生成模型（高保真） + 高效采样 +理论一致

生成模型选择

迭代策略，同时具有完整的误差分析工具

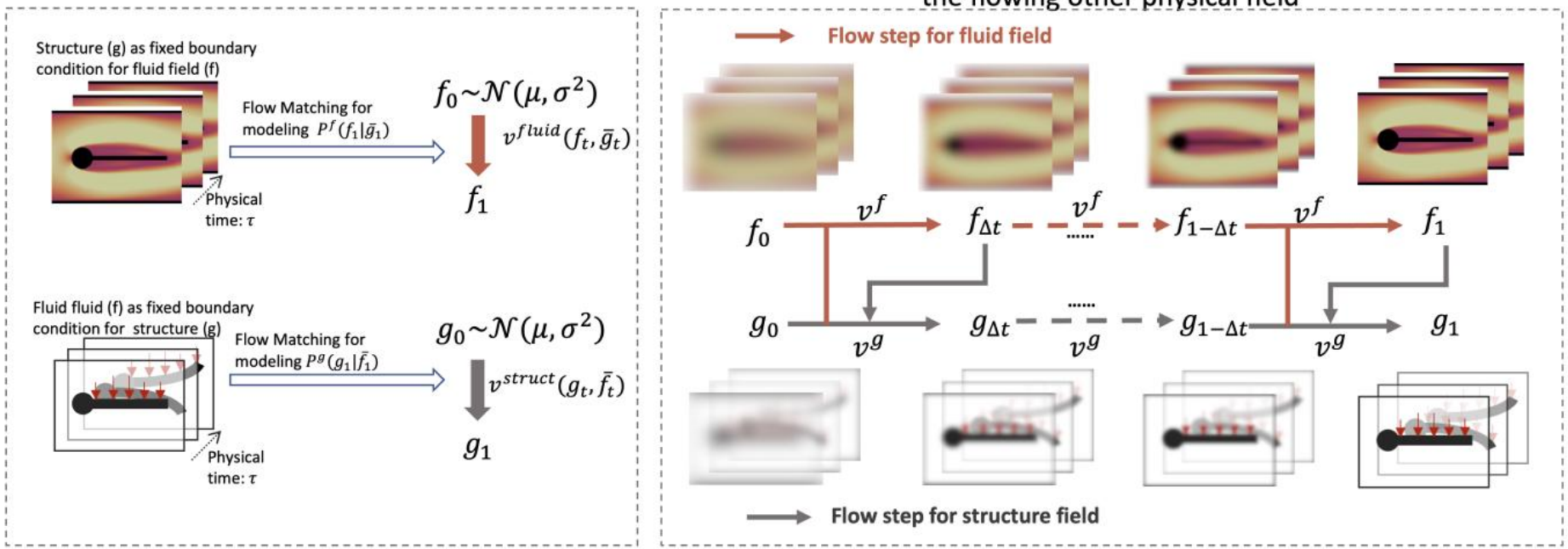
仅用解耦的物理场训练+Functional Flow Matching + Operator Splitting -> 耦合物理场求解-> 详细的误差分析

?

?

Training with data of decoupled physical fields

Inference through iterative updating each physical field using the flowing other physical field



问题1：Operator Splitting高效在了哪里

问题2：什么是Functional Flow Matching（FFM），为什么用FFM，FM不行吗

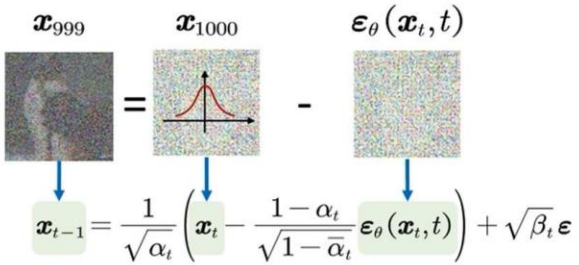
重要Baseline：M2PDE

M2PDE: Compositional Generative Multiphysics and Multi-component PDE Simulation

Tao Zhang[1] Zhenhai Liu[1] Feipeng Qi[1] Yongjun Jiao[1] Tailin Wu[2]

松耦合，基于Diffusion的迭代，有效但迭代方式并无理论证明

DDPM：



$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon_t$$

Zi描述第i个解耦场的状态

条件去噪模型训练:其他解耦场固定作为条件 $L_{\mathrm{MSE}} = ||\epsilon - \epsilon_\theta(\sqrt{1-\beta_s} z_i + \sqrt{\beta_s}\epsilon; z_{\neq i}, s)||_2^2$

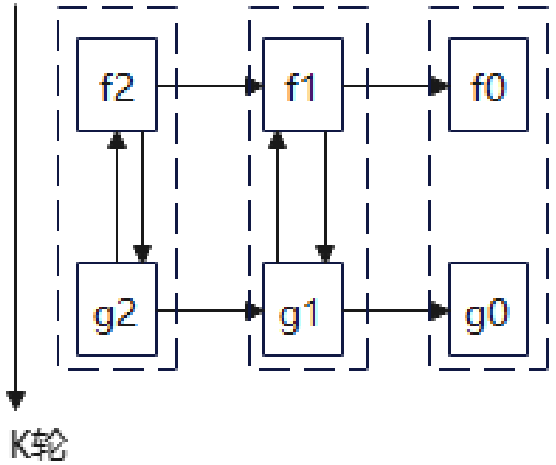
Algorithm 4 Inference of Multiphysics Simulation with the Composition of Diffusion Models.

Require: Compositional set of diffusion models $\epsilon_\theta^i(z_{\neq i}, C, t)$, $i = 1, 2, \ldots, N$, outer inputs C, number of reverse diffusion steps T, number of outer iterations K.

for $k = 1, \ldots, K$ **do**
  $z_T \sim \mathcal{N}(0, I)$
  for $t = T, \ldots, 1$ **do**
    for $i = 1, \ldots, N$ **do**
      $\hat{z}_i^0 = \epsilon_\theta^i(z_{\neq i}^t, C, t)$ ▷ Estimate clean sample for component i
    end for
    Compose $\hat{z}^0 = (\hat{z}_1^0, \ldots, \hat{z}_N^0)$
    $z^{t-1} = p_\theta(z^t | \hat{z}^0, t)$ ▷ Perform conditional sampling step
  end for
  $z^{(k)} = z^0$
end for

Ensure: $z_1^{(K)}, z_2^{(K)}, \ldots, z_N^{(K)}$

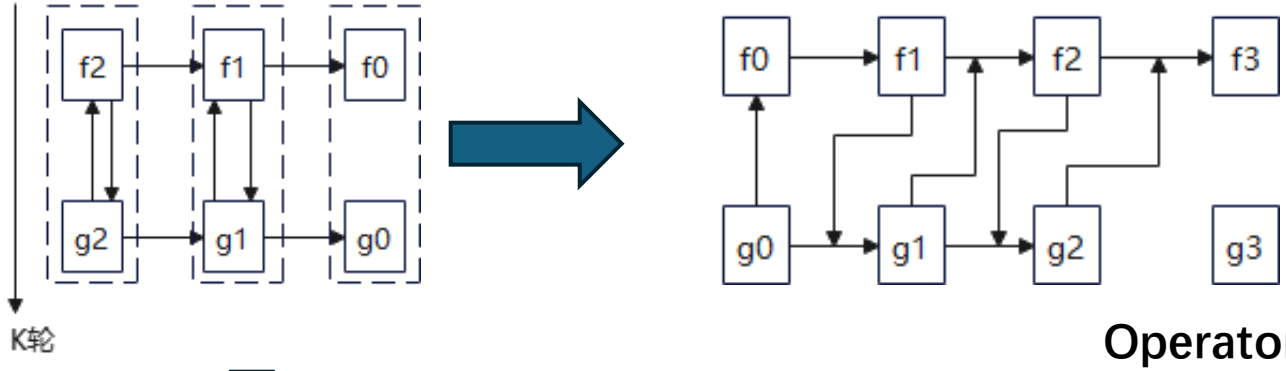
缺点1：每一个扩散时间T是平行的
缺点2：外部还要重新迭代K次
缺点3：并无理论证明

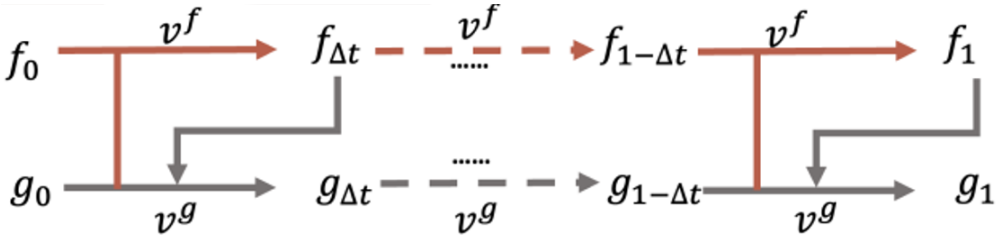


重复K轮的Jacobi迭代法

问题1：Operator Splitting 迭代方法高效在了哪里

改进1：希望优化迭代方法（一步到位，中间迭代交换信息）
改进2：流形学习（flow-based learning），在采样时只需少量的计算步骤（相较于扩散模型的多步反向传播





Operator Splitting：Lie–Trotter Splitting

Algorithm 4 Inference of Multiphysics Simulation with the Composition of Diffusion Models.

Require: Compositional set of diffusion models $\epsilon_\theta^i(z_{\neq i}, C, t)$, $i = 1, 2, \ldots, N$, outer inputs C, number of reverse diffusion steps T, number of outer iterations K.

for $k = 1, \ldots, K$ **do**
  $z_T \sim \mathcal{N}(0, I)$
  for $t = T, \ldots, 1$ **do**
    for $i = 1, \ldots, N$ **do**
      $\hat{z}_i^0 = \epsilon_\theta^i(z_{\neq i}^t, C, t)$ ▷ Estimate clean sample for component i
    end for
    Compose $\hat{z}^0 = (\hat{z}_1^0, \ldots, \hat{z}_N^0)$
    $z^{t-1} = p_\theta(z^t | \hat{z}^0, t)$ ▷ Perform conditional sampling step
  end for
  $z^{(k)} = z^0$
end for
Ensure: $z_1^{(K)}, z_2^{(K)}, \ldots, z_N^{(K)}$

Diffusion

Algorithm 1 Coupled Sampling via Lie–Trotter Splitting

Input: initial $u_0 = (f_0, g_0)$, trained operators $\hat{v}_f, \hat{v}_g$, steps N
$u \leftarrow u_0$
for $k = 0$ to $N - 1$ **do**
  $t \leftarrow k/N$
  $f \leftarrow f + \tau\, \hat{v}_f(f, g, t; \theta_f)$
  $g \leftarrow g + \tau\, \hat{v}_g(f, g, t; \theta_g)$
  $u \leftarrow (f, g)$
end for
return u

Flow Matching

问题2：什么是Functional Flow Matching（FFM），为什么用FFM，FM不行吗

直接看最终形式

Flow Matching VS Functional Flow Matching

数据空间	$\mathbb{R}^n$ 有限维	$\mathcal{F}$ (无限维函数空间)
概率描述	概率密度 $p(x)$	概率测度 μ
噪声分布	$\mathcal{N}(0, I_n)$ 标准高斯)	$\mathcal{GP}(0, C_0))$ 高斯过程
协方差结构	单位矩阵 I_n	协方差算子 $C_0: \mathcal{F} \times \mathcal{F} \to \mathbb{R}$

连续性方程

$$\frac{d}{dt} p_t(x) + \operatorname{div}(p_t(x) v_t(x)) = 0,$$

$$\partial_t \mu_t + \operatorname{div}(v_t \mu_t) = 0 \qquad \text{on } \mathcal{F} \times [0,1].$$

Weak Continuity PDE：函数空间 内的概率分布的演化，它通
与测试函数的内积来进行定义

$$\int_0^1 \int_{\mathcal{F}} (\partial_t \varphi(g,t) + \langle v_t(g), \nabla_g \varphi(g,t)) \, \mathrm{d}\mu_t(g) \, \mathrm{d}t = 0$$

条件路径：

对于数据点 $x_1 \in \mathbb{R}^n$，定义条件概率密度路径 $p_t(x \mid x_1)$：

$$p_t(x \mid x_1) = \mathcal{N}\left(x, \mid \mu_t(x_1) \sigma_t^2 I\right)$$

OT 调度： $\mu_t(x) = t x_1$, and $\sigma_t(x) = 1 - (1 - \sigma_{\min}) t.$

对于数据函数 $f \in \mathcal{F}$，定义条件概率测度路径 μ_t^f：

$$\mu_t^f = \mathcal{N}(m_t^f, C_t^f)$$

$$m_t^f = tf \qquad \sigma_t^f = 1 - (1 - \sigma_{\min}) t.$$

问题2：什么是Functional Flow Matching（FFM），为什么用FFM，FM不行吗

直接看最终形式

Flow Matching VS Functional Flow Matching

生成路径向量场：

$$u_t(x|x_1) = \frac{\sigma_t'(x_1)}{\sigma_t(x_1)}(x - \mu_t(x_1)) + \mu_t'(x_1).$$

$$v_t^f(g) = \frac{(\sigma_t^f)'}{\sigma_t^f}(g - m_t^f) + \frac{\mathrm{d}}{\mathrm{d}t} m_t^f$$

（1）形式相似性：在公式形式上，FM 和 FFM 的条件向量场完全相同，只是：

- FM：$x, x_1 \in \mathbb{R}^n$
- FFM：$g, f \in \mathcal{F}$

(2)训练函数关键区别：

- FM：范数 $\|\cdot\|_{\mathbb{R}^n}$ 是欧几里得范数
- FFM：范数 $\|\cdot\|_{\mathcal{F}}$ 是函数空间范数（如 L^2 范数）

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,q(x_1),p_t(x|x_1)}\left\|v_t(x) - u_t(x|x_1)\right\|^2,$$

$$\mathcal{J}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],f\sim\nu,g\sim\mu_t^f}\left[\left\|v_t^f(g) - u_t(f \mid \theta)\right\|^2\right]$$

(3)模型区别：使用FFM的时候直接用FNO就可以了

- FM：Unet
- FFM：FNO

问题2：什么是Functional Flow Matching（FFM），为什么用FFM，FM不行吗

重要创新点：使用Operator Splitting 在数值分析中充分的误差界限理论工具，保证FFM使用**Lie-Trotter分裂**

理论分析需要在函数空间中进行：

- 多物理场（流速、温度、位移）本质是**定义在连续区域 的函数**，离散网格只是数值近似。
- 函数空间框架提供了**网格无关的收敛性分析**
- 离散化误差可作为额外项单独分析

在FFM使用Lie-Trotter分裂法 中的 **误差界限 ：**

$$W_1\left(\mu_1^{(\tau,\mathrm{learn})},\mu_1\right) \le C_{\mathrm{stab}}\left(\tau+\varepsilon_f+\varepsilon_g\right),$$

Wasserstein-1 距离（W_1）：衡量两个分布之间的误差

- τ是 **Lie-Trotter分裂** 的步长，分裂方法是 **一阶方法**，因此误差随着步长的增大而线性增长。
- ϵ_f和 ϵ_g是学习的条件速度场近似真实条件速度场的误差。
- C_{stab}是由 **Lipschitz 常数** 和 **增长常数** 决定的稳定性常数。

代码中的 Conditional Flow Matching 与 几种常见的 Operator Splitting 方法

```
┌──────────────────────────────────────────────────────────────┐
│                        DataLoader Batch                      │
│   input:  (B, C_in,  T, H, W)                                │
│   target: (B, C_out, T, H, W)                                │
│   attrs:  Dict{field, split, idx}                            │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                data_normalizer.preprocess()                  │
│                归一化处理 (-1 到 1 范围)                       │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                permute 维度变换                               │
│   input:  (B, C_in, T, H, W) → (B, T, H, W, C_in)            │
│   target: (B, C_out,T, H, W) → (B, T, H, W, C_out)           │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│           torch.cat([input, target], dim=-1)                 │
│           joint_state: (B, T, H, W, C_in + C_out)            │
└──────────────────────────────────────────────────────────────┘
                              │
```

```
┌──────────────────────────────────────────────────────────────┐
│          CFM.sample_location_and_conditional_flow()          │
│   x0 = randn(joint_state)                                    │
│   t ~ Uniform(0,1)                                           │
│   xt = (1-t)*x0 + t*joint_state                              │
│   ut = joint_state - x0                                      │
│   输出: t, xt, ut                                             │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                        Model Forward                         │
│   vt = model(xt, t*1000, cond=attrs)                         │
│   输出: (B, T, H, W, C_in + C_out) 预测速度                    │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                        Loss Computation                      │
│   loss = MSE(vt[..., C_in:], ut[..., C_in:])                 │
│   (只计算目标部分的预测误差)                                    │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                   Backward + Optimizer Step                  │
│   loss.backward()                                            │
│   optimizer.step()                                           │
│   EMA update                                                 │
└──────────────────────────────────────────────────────────────┘
```

采样过程中的迭代更新方法

5.3.2 Lie分裂更新（类比算子分裂）

```
# 顺序更新：流体→结构
vt_fluid = model_fluid(x, t, x0, cond)
x_fluid = x + vt_fluid * dt
x_fluid[..., -1:] = x[..., -1:]  # 保持结构场不变
vt_structure = model_structure(x_fluid, t, x0,
cond)
return x_fluid + vt_structure * dt
```

5.3.1 Jacobi同步更新（类比传统Jacobi迭代）

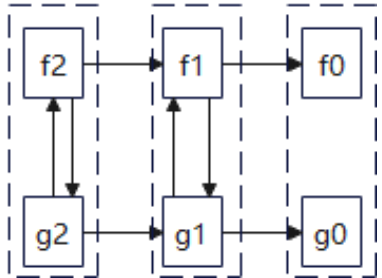
```
# 同时更新所有场
vt_fluid = model_fluid(x, t, x0, cond)
vt_structure = model_structure(x, t, x0, cond)
vt = torch.cat([vt_fluid, vt_structure],
dim=-1)
return x + vt * dt
```

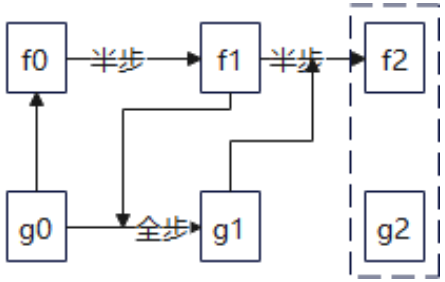
4.1.3 Strang 分裂（半步更新）

```
# 三步分裂
if flag == "strang":
    # 1. 流体半步
    vt_fluid = model_fluid(x, tb, x0, cond)
    x_fluid = x + vt_fluid * dt * 0.5
    x_fluid[..., -1:] = x[..., -1:]

    # 2. 结构全步
    vt_structure = model_structure(x_fluid, tb, x0, cond)
    x_structure = x_fluid + vt_structure * dt
    x_structure[..., :-1] = x_fluid[..., :-1]

    # 3. 流体半步
    vt_fluid_2 = model_structure(x_structure, tb, x0,
cond)
    x = x_structure + vt_fluid_2 * dt * 0.5
    x[..., -1:] = x_structure[..., -1:]
    return x
```





作者的叙事框架

1.把问题定义在了函数空间的Flow Matching：

$$\mathcal{U} := \mathcal{F} \times \mathcal{G}, \|u\|_{\mathcal{U}}^2 = \|f\|_{\mathcal{F}}^2 + \|g\|_{\mathcal{G}}^2.$$

F 和 G 是定义在域 $D \subset \mathbb{R}^d$ 上的分离的希尔伯特空间。为了实现，F和 G可以被具体设定为 $L^2\left(D;\mathbb{R}^{d_f}\right)$和 $L^2\left(D;\mathbb{R}^{d_g}\right)$，这些空间是可分的。

FFM中提出的弱连续性方程——描述在函数空间上的概率测度变化：

$$\int_0^1 \int_{\mathcal{U}} \left(\partial_t \varphi(u) + \langle D\varphi(u), v(t,u)\rangle_{\mathcal{U}}\right) d\mu_t(u)\, dt = 0,$$

$$\int_0^1 \int_{\mathcal{U}} \left(\partial_t \varphi(u) + \langle D\varphi(u), v^{(f)}(t,u)\rangle_{\mathcal{U}}\right) d\mu_t(u)\, dt = 0.$$

$$\int_0^1 \int_{\mathcal{U}} \left(\partial_t \varphi(u) + \langle D\varphi(u), v^{(g)}(t,u)\rangle_{\mathcal{U}}\right) d\mu_t(u)\, dt = 0.$$

$$\mathcal{D}_f = \{(f_0, g_0) \mapsto (f_1, \bar{g})\}, \mathcal{D}_g = \{(f_0, g_0) \mapsto (\bar{f}, g_1)\}$$

如果我们能够从解耦的空间域（数据集）Df和 Dg中分别学习到 v_f和 v_g，它们的和将恢复联合弱演化的生成器。

Flow Matching的速度向量就是相加形式

$$v = v^{(f)} + v^{(g)}$$

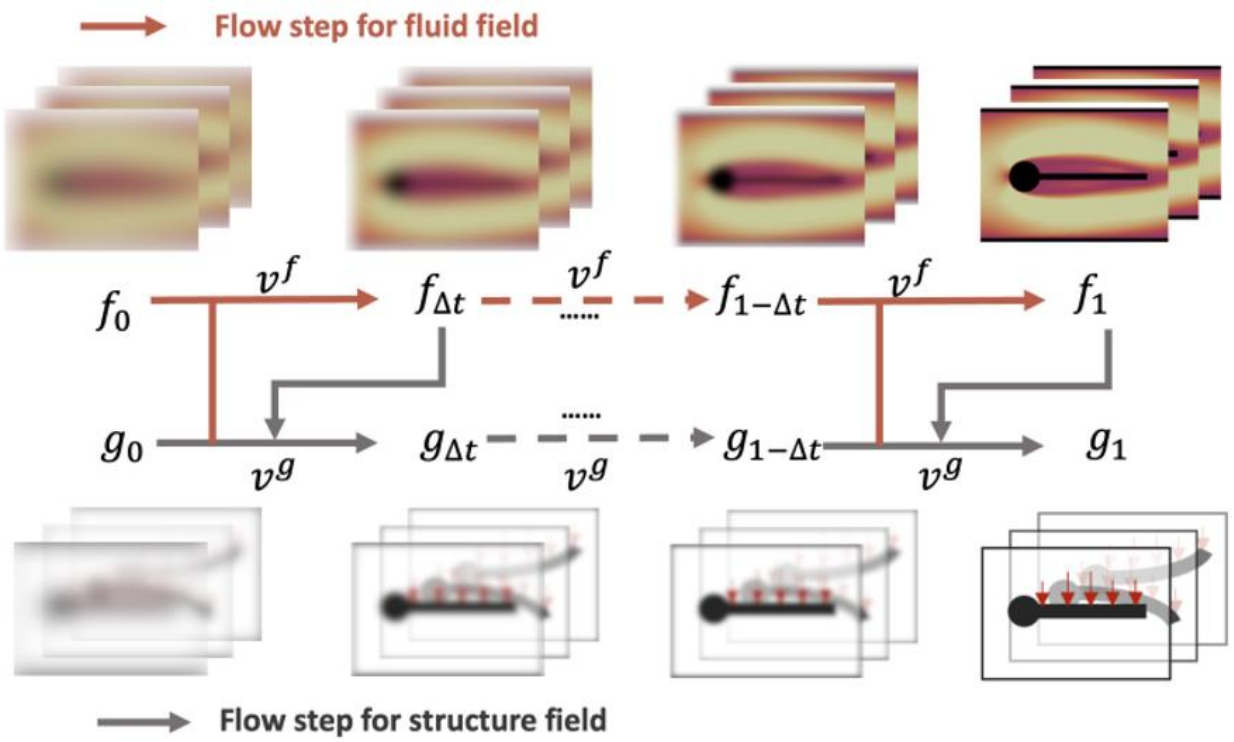
作者的叙事框架

2.定义Operator Splitting的数学公式：

$$S_{t+h\leftarrow t} \approx S^{(g)}_{t+h\leftarrow t} \circ S^{(f)}_{t+h\leftarrow t},$$

定义Flow Map：

$$\mu_{t+h} \approx \left(\Phi^{(g)}_{t+h\leftarrow t} \circ \Phi^{(f)}_{t+h\leftarrow t}\right)_{\#} \mu_t.$$



作者的叙事框架

3.Functional Flow Matching的训练及推理

$$u_t(x|x_1) = \frac{\sigma_t'(x_1)}{\sigma_t(x_1)}(x - \mu_t(x_1)) + \mu_t'(x_1).$$

定义Functional Flow Matching 的最优传输路径，这里σmin=0

$$\mu_t(x) = tx_1, \text{ and } \sigma_t(x) = 1 - (1 - \sigma_{\min})t.$$

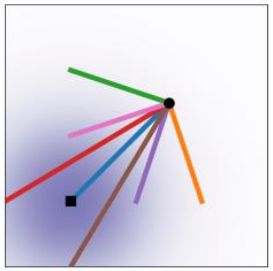
$$\mathcal{D}_f = \{(f_0, g_0) \mapsto (f_1, \bar{g})\}, \mathcal{D}_g = \{(f_0, g_0) \mapsto (\bar{f}, g_1)\}$$

$$\text{Sample } (f_1, \bar{g}) \sim \mathcal{D}_f$$

Zf | Zg即最开始的x0（代码中是高斯噪声）

$$f_t = (1-t)z_f + tf_1 \text{ and } g_t = (1-t)z_g + t\bar{g}.$$

速度向量的真值： $\frac{df_t}{dt} = v_f = f_1 - z_f$



OT

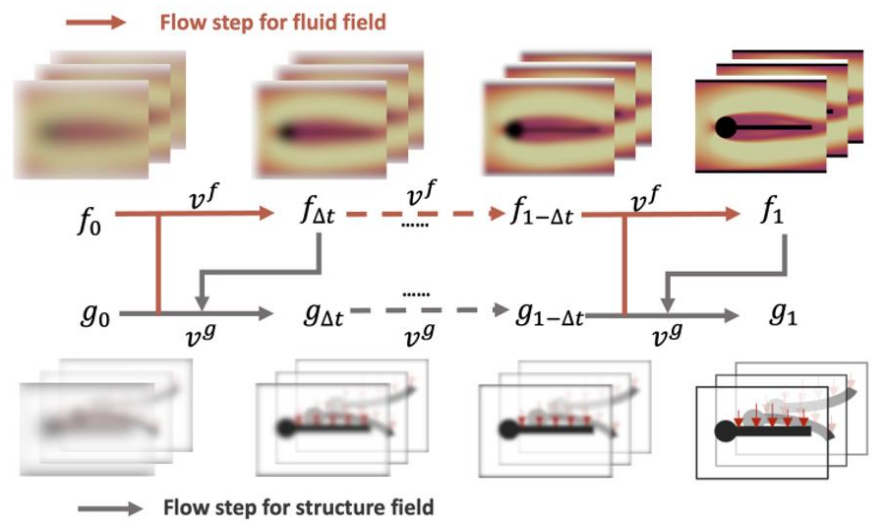
对于两个解构场中间步： 对f训练时，输入g但只对f做MSE

$$\mathcal{L}_f(\theta_f) = \mathbb{E}_{t,(f_1,\bar{g})\sim\mathcal{D}_f,\, z_f, z_g}\left[\|v_f - \hat{v}_f(f_t, g_t, t; \theta_f)\|_{\mathcal{F}}^2\right],$$

$$\mathcal{L}_g(\theta_g) = \mathbb{E}_{t,(\bar{f},g_1)\sim\mathcal{D}_g,\, z_f', z_g'}\left[\|v_g - \hat{v}_g(f_t, g_t, t; \theta_g)\|_{\mathcal{G}}^2\right].$$

Algorithm 1 Coupled Sampling via Lie–Trotter Splitting

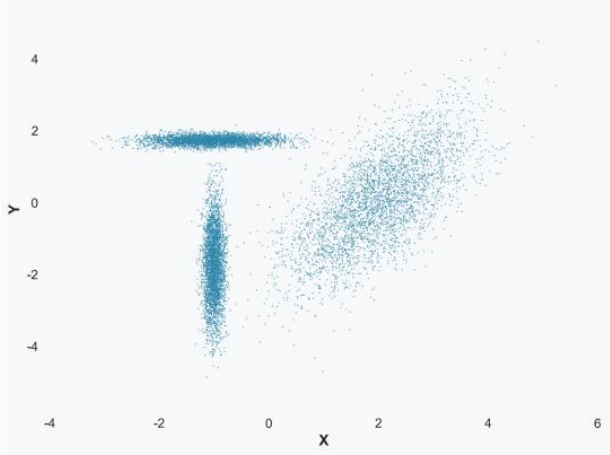
Input: initial $u_0 = (f_0, g_0)$, trained operators $\hat{v}_f, \hat{v}_g$, steps N
$u \leftarrow u_0$
for $k = 0$ to $N - 1$ **do**
    $t \leftarrow k/N$
    $f \leftarrow f + \tau\, \hat{v}_f(f, g, t; \theta_f)$
    $g \leftarrow g + \tau\, \hat{v}_g(f, g, t; \theta_g)$
    $u \leftarrow (f, g)$
end for
return u



实验流程

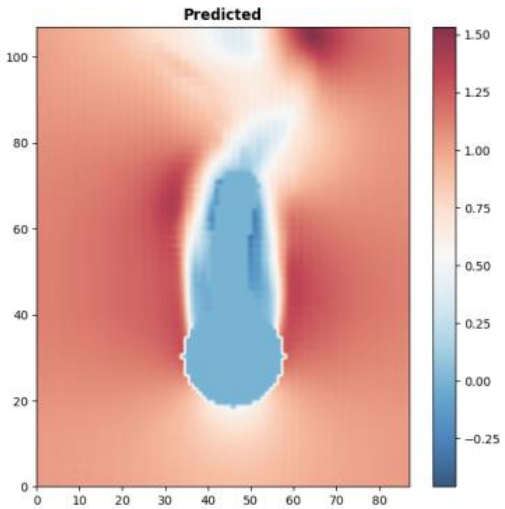
场景选择：简单合成数据 → 经典基准问题 → 复杂多场耦合

耦合模式：解构场单独验证->松耦合验证->紧耦合完整验证

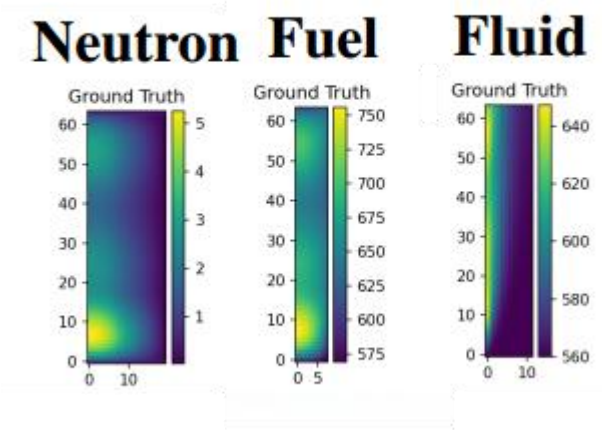


混合高斯场景
证明条件生成机制的有效性

高斯混合模型中，每个成分都是一个**高斯分布**，但是这些成分的**加权组合**构成了最终的混合分布。



流固混合（Double Cylinder, Turek-Hron）
评估模型在经典基准问题上的单场预测精度



核芯热
更加复杂

Baseline 设置：
网络模型：（FNO|CNO的频域空间）

- **M2PDE基线**
- Surrogate模型（迭代收敛方式更特别）

Algorithm 3 Surrogate Model Combination Algorithm for Multiphysics Simulation.

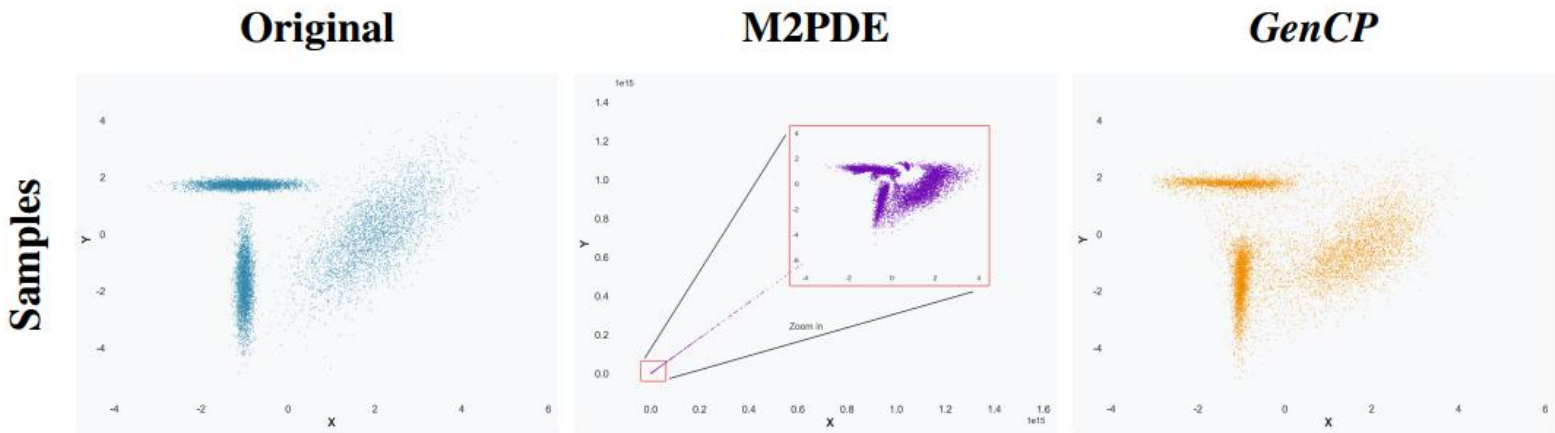
Require: Compositional set of surrogate model $\epsilon_\theta^i(z_{\neq i}, C)$, $i = 1, 2, \ldots, N$, outer inputs C, maximum number of iterations M, tolerance $\epsilon_{\max}$, relaxation factor α.

```
Initialize constant fields z_i, i = 1, ..., N, m = 0
while m < M and ε > ε_max do
    m = m + 1
    for i = 1, ..., N do
        ẑ_i = z_i
        z_i = α ε_θ^i(z_{≠i}, C) + (1 − α) ẑ_i
    end for
    ε = L1(z_i − ẑ_i), i = 1, ..., N
end while
```

Ensure: $z_1, z_2, \ldots, z_N$

混合高斯场景

	Easy Distribution		Complex Distribution	
	GenCP	M2PDE	*GenCP*	M2PDE
W_1	**0.4366**	0.5177	**0.4928**	25450.5442
MMD	**0.0095**	0.0141	**0.0053**	inf
Energy Distance	**0.0411**	0.0625	**0.0061**	332.3619



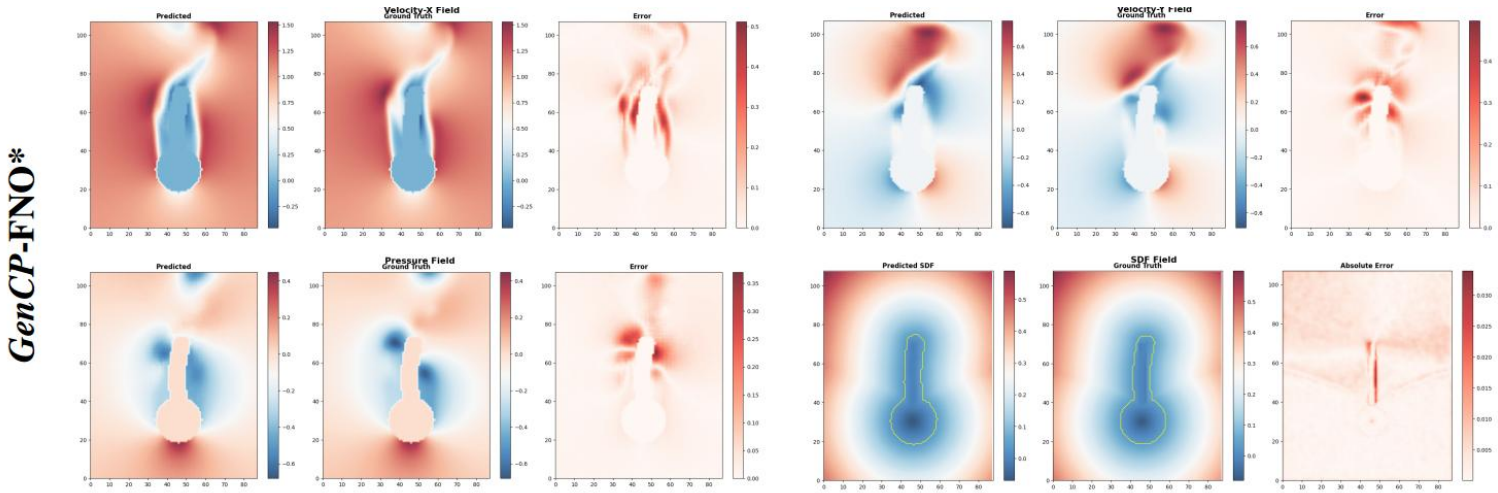
Turek-Hron数据集

•数据内容：

- 流体场：速度（x,y）、压力 共3通道
- 结构场：SDF 1通道
- 空间分辨率：128×88网格

•数据分割：

- flow_condition_on_cylinder/：流体训练数据（条件为结构场）
- cylinder_condition_on_flow/：结构训练数据（条件为流体场）
- couple/：完整耦合数据用于推理验证



对L2误差（Relative L2 Error）：

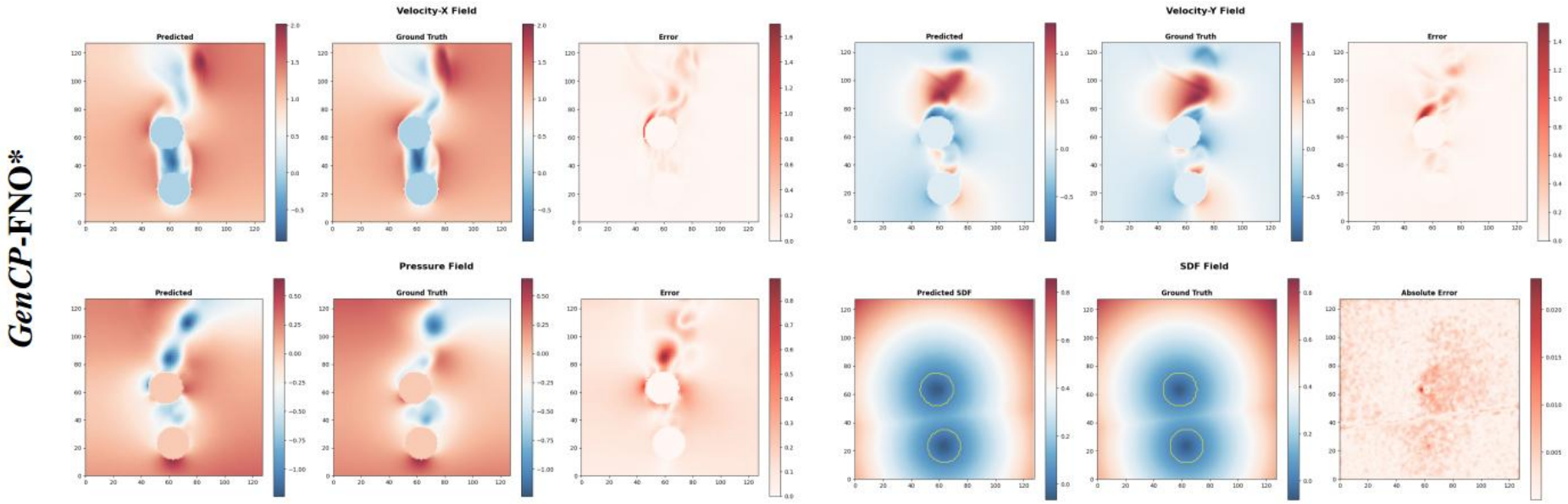
$$\text{Rel}-\text{L2} = \frac{\| u_{pred} - u_{true} \|_2}{\| u_{true} \|_2}$$

Rel L2 Norm Field	Validation on Decoupled Data				Test on Coupled Data				Inference Time
	u	v	p	SDF	u	v	p	SDF	
Joint Training	/	/	/	/	0.0088	0.03441	0.0544	0.0079	/
M2PDE-FNO*	0.0452	0.1582	0.1444	0.0206	0.0590	0.2415	0.2474	0.2482	277.20s
Surrogate-FNO*	0.0181	0.0756	0.0953	0.0087	0.0550	0.2257	0.2553	0.0112	93.20s
Our *GenCP*-FNO*	0.0091	0.0471	0.0548	0.0069	**0.0396**	**0.1678**	**0.1897**	**0.0081**	**19.50s**
M2PDE-CNO	0.0497	0.2075	0.3466	0.03937	0.05769	0.2809	0.4087	0.0390	347.00s
Surrogate-CNO	0.0204	0.0994	0.1526	0.0205	0.0469	0.1888	0.2278	0.0242	300.25s
Our *GenCP*-CNO	0.0150	0.0626	0.1187	0.0152	**0.0388**	**0.1821**	**0.2166**	**0.0183**	**16.25s**

Double Cylinder数据集

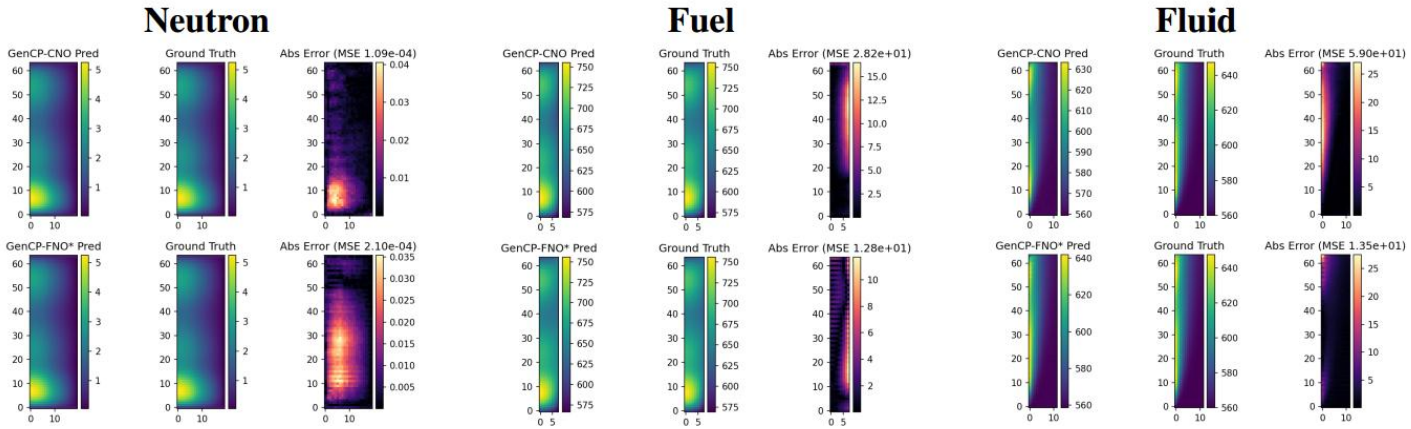
- 来源：流体-结构相互作用基准问题，两个圆柱绕流
- 数据内容：
  - 流体场：速度（x,y）、压力
  - 结构场：圆柱位移场（SDF）
  - 空间分辨率：128×128网格

Rel L2 Norm	Validation on Decoupled Data				Test on Coupled Data			
Field	u	v	p	SDF	u	v	p	SDF
Joint Training	/	/	/	/	0.0110	0.0387	0.0786	0.0034
M2PDE-FNO*	0.0128	0.0414	0.2238	0.0065	0.0571	0.2583	0.8879	0.0152
Surrogate-FNO*	0.0189	0.0625	0.0681	0.0041	0.0717	0.3058	0.4979	0.0196
Our *GenCP*-FNO*	0.0098	0.0441	0.0525	0.0060	**0.0522**	**0.2397**	**0.3987**	**0.0061**
M2PDE-CNO	0.0295	0.2072	0.6468	0.0156	0.0379	0.2694	0.9966	0.0179
Surrogate-CNO	0.0364	0.1078	0.2060	0.0067	0.0612	0.2058	1.0722	0.0080
Our *GenCP*-CNO	0.0141	0.0474	0.1753	0.0058	**0.0279**	**0.1150**	**0.6208**	**0.0055**



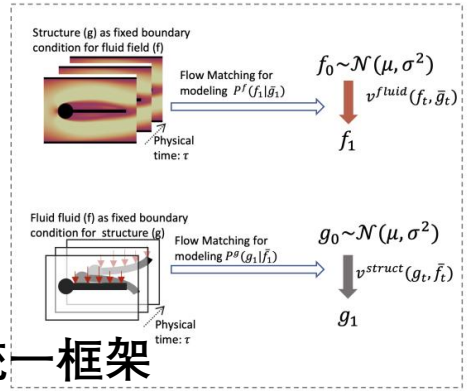
NTcouple数据集（核热耦合）

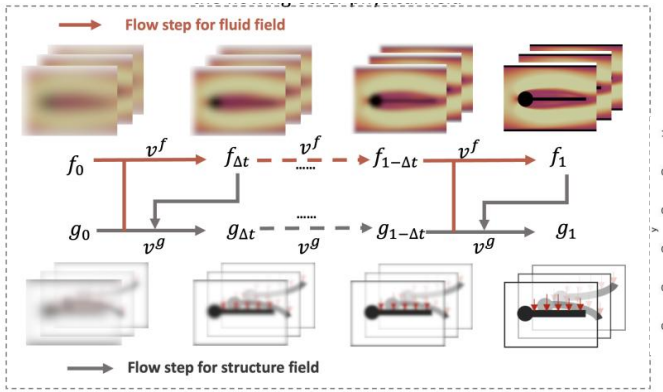
•来源：核反应堆中子-热工-水力耦合模拟
•三个物理场：空间维度：中子场64×20，固体场64×8，流体场64×12

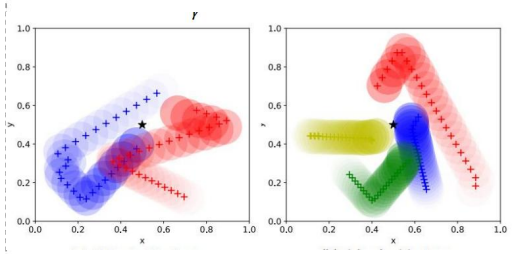


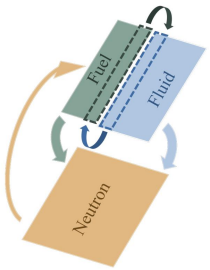
Rel L2 Norm	Decoupled validation on decoupled data			Decoupled validation on coupled data			Coupled test on coupled data		
Field	Neutron	Fuel	Fluid	Neurton	Fuel	Fluid	Neurton	Fuel	Fluid
Our *GenCP*-FNO*	0.0022	0.0006	0.0038	0.0081	0.0371	0.0364	**0.0085**	**0.0364**	**0.0270**
Surrogate-FNO*	0.0086	0.0014	0.0032	0.0140	0.0167	0.0767	0.0149	0.0576	0.1095
M2PDE-FNO*	0.0052	0.0014	0.0018	0.0082	0.0085	0.0237	0.0136	0.1237	0.0463
Our *GenCP*-CNO	0.0024	0.0005	0.0110	0.0047	0.0083	0.0303	**0.0044**	**0.0105**	**0.0330**
Surrogate-CNO	0.0046	0.0007	0.0082	0.0073	0.0044	0.0567	0.0130	0.0553	0.3086
M2PDE-CNO	0.0053	0.0016	0.0092	0.0084	0.0017	0.0236	0.0164	0.0646	0.0401

感想：复杂系统拆解思想







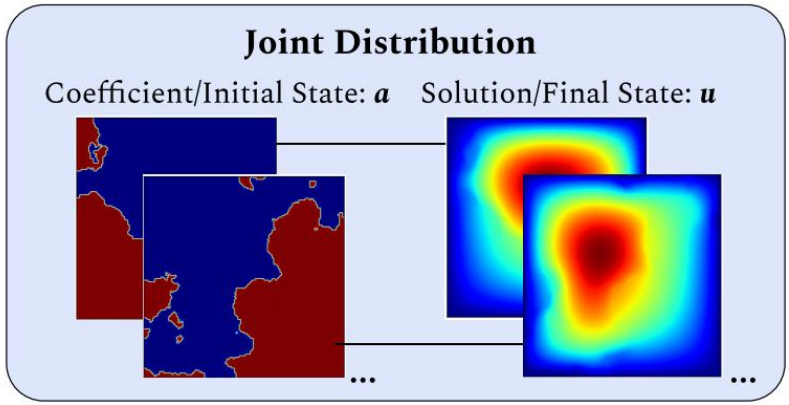


场景1：多参数联合反演的统一框架

当前问题：
传统反问题常单独反演单一参数（如渗透率、源项）
多参数反演时，参数间耦合关系难以建模
不同参数具有不同的物理尺度和不确定性特征
GenCP启发方案：

物理反问题 → 多参数联合状态空间 → 条件生成模型
统一状态空间：将待反演参数拼接为联合状态[θ_1; θ_2; ...; θ_n]
观测条件化：观测数据作为条件信息y，学习p(θ|y)
物理约束集成：PDE残差作为正则化项（类似DiffusionPDE）

物理反问题求解



场景2：多尺度多物理反问题的分层耦合

问题特征：
•物理过程跨越多个时间/空间尺度
•不同尺度需要不同精度的模型
•尺度间耦合机制复杂